

Generative AI - Aided Coding for Robotics Projects

12Sep26 - V2

Prepared by
Dr. Jack Silberman
and
Evan Chou

Department of Electrical and Computer Engineering

University of California, San Diego
9500 Gilman Dr, La Jolla, CA 92093

UC San Diego

JACOBS SCHOOL OF ENGINEERING



UC San Diego

HALICIOĞLU DATA SCIENCE INSTITUTE

Table of Contents

Introduction	3
What is This Guide?	3
Who This Guide Is For	3
What You'll Learn	3
How This Guide Works	4
Prerequisites / Environment Setup	5
Installing Git Locally	5
GitHub Account Setup	6
SSH Key Generation for GitHub	7
Choose your AI Subscription	8
Claude Code	8
ChatGPT Codex	9
GitHub Copilot	10
Cursor	11
Ollama	11
Git/GitHub Fundamentals	13
Key Concepts	13
Git Commands Cheat Sheet	14
Connecting to the Robot	16
Connecting to New Hotspots	16
VS Code Remote SSH Setup	17
Development with Generative AI	19
Prompting Strategies	19
Verify AI-suggested Changes Before Pushing	20
Using Your AI Agent as a Development Partner	21
Debugging Strategies	22
Real Example: Debugging ROS 2 Hardware Integration with Claude Code	25

Introduction

What is This Guide?

Robotics projects are hard. Not because any single piece is impossibly complicated, but because you're simultaneously managing hardware, firmware, operating systems, Python environments, and communication protocols — and when something breaks, the failure could be anywhere in that stack. A servo that doesn't respond might be a wiring issue, a serial permission error, a baud rate mismatch, or a logic bug in code you wrote an hour ago. Figuring out which one is where most project hours disappear.

Generative AI coding agents — tools like Claude Code, GitHub Copilot, Cursor, and Ollama — change what's possible for a student working alone or in a small team. They can read your entire codebase, explain what unfamiliar code does, generate hypotheses about why something is broken, suggest fixes, and help you write better code faster. Used well, they compress the feedback loop between "something is wrong" and "I understand what's wrong and how to fix it" from hours to minutes.

Used poorly — pasting errors in without context, blindly accepting suggestions, treating the agent as a search engine — they waste time and introduce bugs you don't understand.

This guide teaches you how to use them well.

Who This Guide Is For

This guide is written for students in robotics courses at UC San Diego's Jacobs School of Engineering and Halicioğlu Data Science Institute. It assumes you are working on a robotics project involving a Raspberry Pi, real hardware, and Python or ROS 2 code. It does not assume you have prior experience with AI coding tools, Git, or Linux systems.

You don't need to be a software engineer. You need to be curious, willing to read error messages carefully, and open to using AI tools as thinking partners rather than answer machines.

What You'll Learn

This guide covers four things:

How to set up your environment. Git, GitHub, SSH keys, VS Code Remote SSH, and your AI coding agent of choice — everything you need before writing a line of code on your robot.

Git and GitHub fundamentals. Version control is how you avoid losing work, collaborate with teammates, and keep your robot's codebase organized across your laptop and the Pi. This guide covers the concepts and commands you'll actually use.

How to connect to and develop on your robot. Working directly on a Raspberry Pi over SSH — including setting up hotspots for off-campus development — and using VS Code Remote SSH to make the Pi feel like a local machine.

How to use GenAI tools effectively throughout your workflow. Prompting strategies, debugging strategies, using your agent as a development partner, and where LLMs fit into robotics beyond just writing code.

How This Guide Works

The guide is structured as a linear workflow. Each section builds on the last. You'll move between three environments as you work:

- **Your laptop** — where you write code, run your AI agent, and manage Git
- **GitHub** — where your code is backed up and version-controlled in the cloud
- **The Raspberry Pi** — where your code runs on real hardware

Throughout the guide you'll work with an AI coding agent. The guide is tool-agnostic — the strategies work regardless of which agent you pick. You'll choose yours in Section 2.

One note on expectations: AI coding agents are powerful but not infallible. They hallucinate imports, miss edge cases, and sometimes suggest changes that look plausible but introduce subtle bugs. A core theme of this guide is learning to use these tools critically — understanding what they produce, verifying it, and catching mistakes before they become problems on the car.

Prerequisites / Environment Setup

Installing Git Locally

Git is the version control system that tracks every change you make to code. It's how your laptop, GitHub, and the Raspberry Pi all stay in sync. Everything else in this guide depends on it.

First, check if Git is already installed. Open a terminal and run:

```
git --version
```

If you see something like `git version 2.43.0`, skip to the next section. If you get a "command not found" error, install it for your operating system:

macOS: Run the following command. A dialog will appear asking you to install Xcode Command Line Tools, which includes Git. Click Install and wait a few minutes.

```
xcode-select --install
```

Alternatively, if you use Homebrew: `brew install git`

Windows: Download Git from git-scm.com/download/win. During installation, accept the defaults. When the installer asks about line endings, choose "Checkout as-is, commit Unix-style line endings" — this prevents conflicts when your code runs on the Linux-based Raspberry Pi.

Linux (Ubuntu/Debian): `sudo apt update && sudo apt install git`

After installing, tell Git who you are. These lines attach your name and email to every commit. Use the same email as your GitHub account:

```
git config --global user.name "Your Name"
git config --global user.email "you@ucsd.edu"
```

Verify it saved: `git config --global --list`

GitHub Account Setup

GitHub is where the DonkeyCar project lives publicly, where your fork will live, and where you'll submit your pull request.

If you don't have a GitHub account yet:

1. **Create an account.** Go to github.com/signup. Use your UCSD email address (@ucsd.edu) — this is required for student benefits in the next step.
2. **Verify your email.** Check your UCSD inbox for a verification email from GitHub and click the link. You won't be able to push code or open pull requests until your email is verified.
3. **Activate the GitHub Student Developer Pack.** Go to education.github.com/pack and click "Sign up for Student Developer Pack." Verify your UCSD enrollment. This gives you free GitHub Copilot, free private repositories, and credits for other tools used in this guide.
4. **Enable two-factor authentication.** Go to github.com/settings/security and enable 2FA. GitHub requires this for all accounts that push code. Use an authenticator app (Google Authenticator or Authy) rather than SMS.

If you already have a GitHub account with a personal email:

You don't need to create a new account — just add your UCSD email as a secondary address so GitHub can verify your student status.

1. **Add your UCSD email to your account.** Go to github.com/settings/emails and click "Add email address." Enter your @ucsd.edu address and click Add.
2. **Verify the email.** Check your UCSD inbox for a verification email from GitHub and click the link.
3. **Activate the GitHub Student Developer Pack.** Go to education.github.com/pack and click "Sign up for Student Developer Pack." When prompted to select an email for verification, choose your @ucsd.edu address. GitHub will verify your enrollment and activate the pack on your existing account.
4. **Enable two-factor authentication if you haven't already.** Go to github.com/settings/security and check whether 2FA is enabled. If not, enable it now using an authenticator app (Google Authenticator or Authy) rather than SMS.

Note: The Student Developer Pack is tied to your account, not your email address. Once activated, all benefits — including free GitHub Copilot — apply regardless of which email you use to sign in day-to-day.

SSH Key Generation for GitHub

SSH keys let your laptop authenticate with GitHub without typing a password every time you push code. You generate a key pair: a private key that stays on your machine, and a public key that you give to GitHub.

⚠ Important: Never share your private key (`id_ed25519`) with anyone. The public key (`id_ed25519.pub`) is the one you'll paste into GitHub — that's the one that's safe to share.

Step 1 — Check for an existing key.

You might already have one from a previous class. Run:

```
ls ~/.ssh
```

If you see `id_ed25519` and `id_ed25519.pub`, skip to Step 3. If the folder doesn't exist or is empty, continue to Step 2.

Step 2 — Generate a new key.

Replace the email with your GitHub email address:

```
ssh-keygen -t ed25519 -C "you@ucsd.edu"
```

When prompted for a file location, press Enter to accept the default. When prompted for a passphrase, you can press Enter to skip it or set one for extra security.

Step 3 — Start the SSH agent and add your key.

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

On macOS, add `--apple-use-keychain` so the key persists across reboots:

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

Step 4 — Copy your public key.

```
cat ~/.ssh/id_ed25519.pub
```

On macOS you can pipe it directly: `pbcopy < ~/.ssh/id_ed25519.pub`

On Windows (Git Bash): `clip < ~/.ssh/id_ed25519.pub`

Step 5 — Add the key to GitHub.

Go to github.com/settings/keys and click "New SSH key." Give it a title like "My laptop," paste your public key, and click "Add SSH key."

Step 6 — Test the connection.

```
ssh -T git@github.com
```

You should see: *Hi <your-username>! You've successfully authenticated...* If you see a warning about the host's authenticity, type **yes** and press Enter.

Choose your AI Subscription

You'll use an AI coding agent throughout this guide to help you read unfamiliar code, suggest changes, and debug errors. Pick one and follow only that agent's setup instructions — they all accomplish the same goal, so choose based on what you already have access to or prefer.

Recommendation: Claude Code is the best fit for this guide because it understands your entire codebase at once, not just the file you have open. GitHub Copilot is the easiest starting point if you want something invisible inside VS Code.

Claude Code

Claude Code is a terminal-based agent by Anthropic that can read your entire repo, run commands, edit files, and explain code in context.

Terminal setup:

1. Install Node.js 18 or later from nodejs.org (choose the LTS version). Verify: `node --version`
2. Install Claude Code globally: `npm install -g @anthropic-ai/claude-code`
3. Navigate to your project folder and launch it. You'll be prompted to log in with your Anthropic account on first run: `claude`
4. Test it by typing: `What Python version is on this machine?`

VS Code setup:

There is an official Claude Code extension by Anthropic available directly in the VS Code Marketplace. This is the recommended way to use Claude Code inside VS Code — it gives you a sidebar panel with inline diffs, so you can see exactly what Claude wants to change before it touches any file.

1. Open VS Code and go to the Extensions panel (**Ctrl+Shift+X** / **Cmd+Shift+X**). Search for **Claude Code** and install the extension published by Anthropic. Alternatively, open Quick Open (**Ctrl+P** / **Cmd+P**) and paste: `ext install anthropic.claude-code`
2. Open your DonkeyCar project folder in VS Code using File → Open Folder. Claude Code needs the full project root as its working context — opening a single file won't give it codebase-wide awareness. [Claude Code Docs](#)
3. Click the Spark icon (✱) in the Activity Bar on the left sidebar to open the Claude Code panel. On first launch, a sign-in screen appears — click Sign in and complete authorization in your browser using your Anthropic account. [Claude Code Docs](#)
4. Once signed in, you can open Claude Code from several places: the Spark icon in the top-right of the editor toolbar (only appears when a file is open), the Activity Bar icon (always visible), or the Command Palette (**Cmd+Shift+P** / **Ctrl+Shift+P**) by typing "Claude Code". [Claude Code Docs](#)
5. Test it by typing in the panel: `What does the vesc.py file do?` — Claude Code will read the file and explain it in context.

Note: The extension bundles its own copy of the CLI, so you don't need to install the terminal CLI separately to use the VS Code extension. However, if you want to use both the sidebar and the integrated terminal (which is useful for running the car), the two share the same session and auth state — no penalty for switching between them.

ChatGPT Codex

ChatGPT Codex is OpenAI's cloud-based coding agent. You upload files or paste code, ask questions, and get explanations or suggested changes.

Web setup:

1. Go to chat.openai.com and create an account. The free tier gives access to GPT-4o mini; ChatGPT Plus (\$20/mo) unlocks GPT-4o and Codex.
2. Upload individual DonkeyCar Python files using the paperclip icon and ask questions like: "What does this file do?" or "Where is the servo angle being set?"
3. To access Codex specifically, look for "Codex" or "o3" in the model selector at the top of a chat (requires ChatGPT Plus).

VS Code setup:

Install the ChatGPT extension ([openai.chatgpt](#)) from the Extensions panel. Add your OpenAI API key in the extension settings. This adds a chat panel inside VS Code that can reference your open files.

Note: Codex doesn't have persistent access to your file system — you need to paste or upload code each session. For large codebase questions, Claude Code or Cursor will be more efficient.

GitHub Copilot

GitHub Copilot provides inline autocomplete and a chat panel built directly into VS Code. It's free with the GitHub Student Developer Pack.

VS Code setup:

1. Activate Copilot via the Student Developer Pack at education.github.com/pack. Verify your UCSD enrollment — Copilot becomes free on your account once approved.
2. In VS Code, open Extensions (**Ctrl+Shift+X** / **Cmd+Shift+X**) and install both [GitHub.copilot](#) and [GitHub.copilot-chat](#).
3. VS Code will prompt you to sign in with GitHub. Complete the OAuth flow in your browser. The Copilot icon will appear in the status bar when active.
4. Test it: open any `.py` file and start typing a function. Copilot suggests completions in gray text — press Tab to accept. Open chat with **Ctrl+Alt+I**.

Terminal setup:

Install the Copilot CLI to bring autocomplete into the terminal:

```
npm install -g @githubnext/github-copilot-cli
github-copilot-cli auth
```

Use `??` to ask shell questions and `git?` for Git-specific help:

```
?? how do I undo my last commit without losing changes
```

Cursor

Cursor is a VS Code fork with AI built into every panel — inline completions, a chat sidebar, and a selection edit shortcut (**Cmd+K**). Free tier available.

VS Code (Cursor) setup:

1. Download Cursor from cursor.com for your OS. It's a standalone app but all VS Code extensions work inside it.
2. On first launch, import your VS Code extensions, keybindings, and settings — including the Remote SSH extension you'll need in Section 5.
3. Create a Cursor account, sign in, and choose a model in Settings → Models. The free tier includes GPT-4o and Claude Sonnet.
4. Test it: open any file, press **Cmd+L** / **Ctrl+L** to open the chat sidebar, and ask a question about the file. Select any code block and press **Cmd+K** to ask Cursor to rewrite or explain it inline.

Terminal setup:

Cursor's integrated terminal works identically to VS Code's. There's no separate Cursor CLI. For terminal-first workflows, pairing Cursor with Claude Code is a strong combination.

Ollama

Ollama lets you run a local LLM on your laptop — no internet, no API key, no data leaving your machine. Requires at least 8 GB RAM; 16 GB recommended for capable coding models.

Terminal setup:

1. Install Ollama from ollama.com, or on macOS/Linux: `curl -fsSL https://ollama.com/install.sh | sh`
2. Pull a coding model. `qwen2.5-coder:7b` runs on 8 GB RAM and is capable for code explanation. `qwen2.5-coder:32b` needs ~20 GB but is significantly stronger: `ollama pull qwen2.5-coder:7b`
3. Run it: `ollama run qwen2.5-coder:7b`
You'll get an interactive prompt. Paste code snippets and ask questions. Type `/bye` to exit.

VS Code setup:

Install the Continue extension (`continue.continue`) from the Extensions panel. Continue connects to your locally running Ollama instance and adds a chat panel and inline completions to VS Code.

1. Install Continue in VS Code: search `continue.continue` in Extensions.
2. Open the Continue sidebar, click the settings icon, and add Ollama as a provider. Select your pulled model (`qwen2.5-coder:7b`).
3. Make sure Ollama is running in the background (`ollama serve`), then test by opening a DonkeyCar file, highlighting a function, and pressing `Cmd+L` / `Ctrl+L`.

Performance note: Local models are slower than cloud-based ones — expect a few seconds per response on a typical laptop. On Apple Silicon Macs, Ollama uses the Neural Engine and performs well even on smaller machines.

Git/GitHub Fundamentals

Key Concepts

Git is software that runs on your laptop. It tracks every change you make to a codebase over time, letting you save snapshots of your work (called commits), experiment on separate lines of development (called branches), and roll back to any previous state if something breaks. Think of it like a detailed undo history for your entire project — one that you control deliberately.

GitHub is a website that hosts Git repositories in the cloud. It's where teams share code, propose changes, and review each other's work. When you push code to GitHub, you're uploading your local Git history to a remote copy that others can see and collaborate on.

Repository (repo): The full project folder — code, history, and configuration — tracked by Git. DonkeyCar's repository lives at github.com/autorope/donkeycar. Your own copy will live at github.com/<your-username>/donkeycar.

Fork: A personal copy of someone else's repository on GitHub. When you fork DonkeyCar, you get your own version of the entire codebase under your GitHub account. You can make any changes you want to your fork without affecting the original project.

Clone: Downloading a repository from GitHub to your local machine. After cloning, you have a full local copy of the repo — including its entire history — that you can edit with VS Code.

Branch: A parallel version of the codebase where you can make changes without touching the main code. Think of it like a working draft. You'll always create a new branch for your changes rather than working directly on `main`.

Commit: A saved snapshot of your changes. Every commit has a message describing what changed and why. Your commit history is a log of every decision you made.

Push: Uploading your local commits to GitHub. After committing changes on your laptop, you push them so they appear in your remote fork on GitHub.

Pull: Downloading changes from GitHub to your local machine. You'll pull when the DonkeyCar maintainers update the main repo and you want to sync your fork.

Pull Request (PR): A formal proposal to merge your branch into another repository — in this case, the main DonkeyCar repo. Maintainers review the changes, leave comments, and either approve or request revisions before merging.

Upstream: The original repository you forked from. In this guide, "upstream" always refers to github.com/autorope/donkeycar. Keeping your fork in sync with upstream means pulling the latest changes from the original project into your own copy.

Git Commands Cheat Sheet

Getting code

`git clone <url>` — Download a repository to your machine

`git fetch upstream` — Check for new changes in the upstream repo without applying them

`git pull` — Download and apply the latest changes from the remote branch you're on

Branches

`git branch` — List all branches; the one with a `*` is your current branch

`git checkout -b <branch-name>` — Create a new branch and switch to it

`git checkout <branch-name>` — Switch to an existing branch

Saving changes

`git status` — See which files you've changed since your last commit

`git diff` — See exactly what changed line-by-line in modified files

`git add <file>` — Stage a file, marking it to be included in the next commit

`git add .` — Stage all changed files at once

`git commit -m "Your message here"` — Save a snapshot of all staged changes with a description

Syncing

`git push origin <branch-name>` — Upload your branch to your GitHub fork

`git push -u origin <branch-name>` — Same, but sets the remote as the default for future pushes (use this the first time)

Tip: If you can't remember a command, ask your AI agent: "What's the Git command to [do what you want]?" This is one of the best uses of an AI coding agent — Git syntax is hard to memorize and easy to look up.

Connecting to the Robot

Connecting to New Hotspots

The Raspberry Pi on your DonkeyCar is pre-configured to connect to the [UCSDrobocars2018](#) Wi-Fi network on campus. When you're in the lab, your laptop joins the same network and can reach the Pi directly by its IP address.

However, if you want to develop and test at home or off-campus, you'll need to add your own hotspot to the Pi's list of known networks. To use the car off campus, you need to add your home Wi-Fi or phone hotspot to the Pi's known networks while you still have access to the Pi on campus. Do this before leaving the lab.

You have two options depending on what's most convenient.

Option A — Add your network via the Pi's terminal (recommended)

While connected to the Pi on campus over SSH, run the following command. Replace the values with your own network name and password:

```
sudo nmcli device wifi connect "YourNetworkName" password "YourPassword"
```

To confirm the network was saved:

```
nmcli connection show
```

You should see your network listed alongside [UCSDrobocars2018](#). The Pi will now automatically connect to whichever known network is in range when it boots.

If you're adding a phone hotspot, enable the hotspot on your phone first so the Pi can detect it during the scan. The network name and password are set in your phone's hotspot settings (Settings → Personal Hotspot on iPhone; Settings → Network & Internet → Hotspot on Android).

Option B — Edit the network configuration file directly

If `nmcli` isn't available on your Pi, you can add the network by editing the `wpa_supplicant` config file:

```
sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
```

Add the following block at the bottom of the file, replacing the values with your network details:

```
network={
  ssid="YourNetworkName"
  psk="YourPassword"
  priority=1
}
```

The `priority` field controls which network the Pi prefers when multiple known networks are in range. Higher numbers = higher priority. If you want the Pi to prefer your home network over `UCSDrobocars2018` when both are available, set your home network's priority to `2`. To keep the campus network as default, leave your home network at `1`.

Save the file with `Ctrl+O`, then `Enter`, then exit with `Ctrl+X`. Reboot the Pi for the changes to take effect:

```
sudo reboot
```

VS Code Remote SSH Setup

VS Code Remote SSH lets you open the Raspberry Pi's file system directly inside VS Code on your laptop — as if the Pi's files were local. You get full syntax highlighting, your extensions, and an integrated terminal that runs commands on the Pi, all from your familiar editor.

Step 1 — Install the Remote-SSH extension.

Open VS Code and go to the Extensions panel (`Ctrl+Shift+X` / `Cmd+Shift+X`). Search for `Remote - SSH` and install the extension published by Microsoft (`ms-vscode-remote.remote-ssh`).

Step 2 — Add your robot as a new host.

Press `Ctrl+Shift+P` / `Cmd+Shift+P` to open the Command Palette. Type `Remote-SSH: Add New SSH Host` and press Enter.

When prompted for the SSH command, type:

```
ssh pi@ucsdrobocars-DSC-T#
```

Replace `#` with your team number. For example, team 4 would enter:

```
ssh pi@ucsdrobocars-DSC-T4
```

Step 3 — Save the host configuration.

Press Enter when prompted to select an SSH config file to save to. Choose the default option (typically `~/.ssh/config` on macOS/Linux or `C:\Users\<you>\.ssh\config` on Windows). This saves your robot's connection details so you don't have to type them again.

Step 4 — Connect to the host.

Press `Ctrl+Shift+P` / `Cmd+Shift+P` again and type `Remote-SSH: Connect to Host`. Select `ucsdrobotcar-DSC-T#` from the list. A new VS Code window will open and begin connecting.

Step 5 — Enter the password.

The new window will prompt you for the Pi's password in a bar at the top of the screen. Enter the password provided by your instructor and press Enter.

Step 6 — Wait for the connection to establish.

The first time you connect, VS Code installs a small server component on the Pi. This takes a few minutes. You'll see "Opening Remote..." in the bottom-left corner — wait until it shows the host name before doing anything else.

Step 7 — Open your forked repo folder.

Once connected, go to File → Open Folder. VS Code is now browsing the Pi's file system. Navigate to the directory where you cloned your DonkeyCar fork in the previous section and click OK.

The VS Code file explorer will now show the Pi's DonkeyCar files. Any edits you make here happen directly on the Pi — no file transfer needed. Open the integrated terminal with `Ctrl+``` / `Cmd+``` to confirm you're on the Pi:

```
hostname
```

The output should show the Pi's hostname, not your laptop's.

Development with Generative AI

Prompting Strategies

AI coding agents are not search engines. The quality of what they produce depends almost entirely on the quality of the context you give them. These strategies apply regardless of which agent you chose in Section 2.

Give the agent the full file, not just the error.

Pasting only an error message forces the agent to guess at context. Instead, paste the full file (or at least the relevant class) alongside the error. "Here's the full `vesc.py` file and the error I'm seeing when I run the car — what's causing it?" produces much better results than "why am I getting a `KeyError`?"

Tell the agent what the code is supposed to do.

Agents reason better when they understand intent. "This function is supposed to clamp the servo angle between -1 and 1 before sending it to the VESC, but it's currently allowing values outside that range" gives the agent a target to reason toward, not just a symptom to react to.

Ask for explanation before asking for a fix.

When you're in unfamiliar code, ask the agent to explain what a function does before asking it to change anything. "What does the `run()` method in this class do, and what are the expected types of its inputs and outputs?" Then, once you understand it: "Given that, how would you fix the clamping issue?"

Request the minimal change.

Ask for the smallest possible edit that fixes the problem. "What is the minimal change to this function that would fix the clamping bug without changing any other behavior?" This keeps diffs small, makes pull requests easier to review, and reduces the chance of introducing unintended side effects.

Ask the agent to explain its own suggestion.

Before accepting any generated code, ask: "Walk me through what this change does line by line." If the agent can't explain it clearly, or if the explanation reveals the change does something you didn't intend, don't use it.

Use the agent for rubber duck debugging.

Sometimes you don't need the agent to write code — you need to think out loud. Describe the bug to the agent as if you're explaining it to a colleague: "The car is pulling left on straight sections. The steering output looks correct in the logs, but the servo seems to be receiving a different value. Here's

the relevant code..." The act of articulating the problem clearly often surfaces the answer before the agent even responds.

Verify AI-suggested Changes Before Pushing

This is the most important habit to build: never paste AI-generated code into the codebase without reading and understanding every line first.

This isn't just about catching bugs. It's about being able to explain your change when a DonkeyCar maintainer asks about it in a pull request review. If you can't explain why each line is there, you won't be able to respond to feedback, and your PR will stall.

A practical review process for each suggestion:

1. Read it line by line before running it.

Go through the suggestion and ask yourself: do I know what each line does? If not, highlight the lines you don't understand and ask the agent to explain them individually.

2. Compare it to the original.

Look at what changed. Is the diff small and targeted, or did the agent rewrite large sections of code you didn't ask it to touch? Large unrequested rewrites are a red flag — ask for a more minimal version.

3. Check the imports.

AI agents frequently suggest imports that don't exist in the project's dependencies, belong to a different Python version, or are already imported elsewhere in the file. Check every new import statement the agent introduces.

4. Run it and read the output.

After applying the change, run the relevant part of the code and read the terminal output carefully. A change that runs without errors isn't necessarily correct — verify that the behavior is what you actually intended.

5. Ask the agent to review its own suggestion.

After generating a change, prompt: "Are there any edge cases this doesn't handle? Are there any ways this could break existing behavior?" Agents are often better at critiquing their own output than producing perfect code on the first attempt.

Using Your AI Agent as a Development Partner

The prompting strategies in Section 7 cover how to use your AI agent for a specific change. This section is about something broader — how to use your agent as a persistent development partner across your entire workflow, from the moment you sit down to work to the moment you push your final commit.

The distinction matters because most students use AI agents reactively — they hit an error, paste it in, get a fix, move on. That works, but it captures maybe 20% of what the agent is actually capable of in a development context. The students who get the most out of these tools use them proactively, as a thinking partner throughout the entire session, not just a stack overflow replacement.

Start every session with a context dump.

Before writing a single line of code, open your agent and give it the full picture of what you're working on. Paste the relevant files, describe what you're trying to do, and explain where you left off last session. Something like:

"I'm working on a contribution to the DonkeyCar open source project. Here are the relevant files. I'm trying to fix a bug where the VESC driver allows steering values outside -1 to 1. Yesterday I traced the issue to the `run()` method in `vesc.py` but haven't made the fix yet. Help me think through the right approach before I start editing."

This front-loads the context so every subsequent exchange in that session is grounded. Without this, you'll spend half your session re-explaining the same background on every prompt.

Use it to plan before you code.

Before touching any file, ask your agent to walk you through the change at a design level. "What's the minimal set of changes needed to fix this? What are the risks? What existing behavior could this break?" Planning with the agent before coding catches design problems that would otherwise only surface after you've already written the wrong thing.

Ask it to anticipate review feedback.

Before committing, ask: "If you were a DonkeyCar maintainer reviewing this PR, what would you push back on?" This surfaces issues — missing edge case handling, inconsistent style, scope creep — that you're too close to the code to see yourself.

Keep it in the loop when you go off-script.

When something unexpected happens — the Pi behaves differently than your laptop, a test fails that shouldn't, the car does something weird — narrate it to the agent immediately rather than trying to debug silently first. "I just ran the manage script and the car immediately veered hard left before I touched the controller. Here's the terminal output. What are the possible causes?" Narrating the

unexpected is often faster than methodical debugging because the agent can quickly narrow the hypothesis space.

Debugging Strategies

Debugging on a Raspberry Pi connected to real hardware is different from debugging a Python script on your laptop. The feedback loop is slower, errors can have physical consequences, and the interaction between software and hardware introduces failure modes that don't show up in local testing. This section covers a structured approach to debugging in this environment, with your AI agent as a core part of the process.

Read the Full Traceback Before Doing Anything Else

When DonkeyCar crashes, Python prints a traceback. The single most common debugging mistake is reading only the last line — the exception message — and immediately pasting that into the agent.

The last line tells you what went wrong. The lines above it tell you how the code got there. Without the full call stack, the agent is guessing at context. Always paste the entire traceback.

Read it yourself first, from the bottom up:

- The bottom line is the exception type and message
- The lines immediately above it are the function where it crashed and the line number
- The lines above those show the chain of calls that led there

Ask yourself: does the crash location make sense given what I just changed? Is this in my code or in a DonkeyCar internal? Is the exception type what I'd expect? Answering these questions yourself before involving the agent makes you a better debugger and often surfaces the answer before you even finish typing the prompt.

The Hypothesis-First Debugging Loop

When you paste an error into your agent, don't just ask "how do I fix this?" Ask it to generate hypotheses first:

"Here's the full traceback and the relevant code. Before suggesting a fix, give me three possible root causes ordered by likelihood. For each one, tell me how I would verify whether that's actually the problem."

This forces a diagnostic step before a solution step. It also gives you a mental model of the problem space so that when you test each hypothesis and one of them is wrong, you understand why rather than just trying the next suggestion blindly.

Work through hypotheses one at a time. Change one thing, test, observe. If you change three things between test runs you won't know which one fixed it — or which one introduced a new problem.

Hardware vs. Software Triage

A lot of time gets lost on the Pi trying to debug in software what is actually a hardware problem, and vice versa. Before diving into code, run through this triage mentally:

Is the problem consistent or intermittent?

Consistent failures — same error every run — are almost always software. Intermittent failures — works sometimes, crashes sometimes — often point to hardware: a loose cable, a marginal power supply, a serial connection dropping under load. Intermittent failures are significantly harder to debug and the agent is less useful for them because the symptoms aren't reproducible.

Did it ever work?

If the code worked yesterday and doesn't today, and you didn't change anything, suspect hardware or environment first — a cable came loose, the SD card is corrupting, a dependency updated. If the code never worked, suspect your change.

Is the error in your code or in DonkeyCar internals?

Look at the traceback. If the crash is in a file you didn't touch, your change is probably passing a bad value into an existing function rather than having a bug in the function itself. Look at what your code is passing in, not at the function that's receiving it.

Quick hardware checks before debugging software:

```
ls /dev/ttyACM*      # VESC connected over USB serial
```

```
ls /dev/video*       # Camera recognized
```

```
vcgencmd get_throttled # Pi thermal throttling (0x0 = healthy)
```

```
free -h             # Available memory
```

```
df -h               # Disk space on SD card
```

Paste the output of any of these that looks suspicious into your agent along with your error. Hardware state context dramatically improves the quality of the agent's diagnosis.

Debugging Serial and Hardware Communication

If your change touches the VESC, servo, or any hardware interface, a whole category of errors becomes possible that pure software debugging won't catch. These are the most common:

Baud rate and port mismatches. The VESC communicates over serial at a specific baud rate. If the port or rate in your code doesn't match what the VESC expects, you'll get silent failures or garbled data — not a clean Python exception. Ask your agent: "How do I verify that my serial connection to the VESC is actually receiving and sending valid data, not just opening without error?"

Timing issues. The Pi's GPIO and serial interfaces have timing constraints. Code that works at low speeds may fail at higher throttle or steering rates because commands are arriving faster than the hardware can process them. If behavior degrades under load but works at low speed, mention this explicitly to the agent — it changes the diagnosis entirely.

Permission errors on device files. If you see a `PermissionError` when opening `/dev/ttyACM0` or similar, the Pi user doesn't have permission to access the device. Fix with:

```
sudo usermod -aG dialout $USER
```

Log out and reconnect for it to take effect.

Reading Logs When There's No Traceback

Sometimes the car crashes without printing anything useful — the process exits silently or the terminal output scrolls past before you can read it. A few techniques for recovering useful information:

Pipe output to a log file while driving:

```
cd ~/mycar
```

```
python3 manage.py drive 2>&1 | tee drive_log.txt
```

The `tee` command prints to the terminal and writes to a file simultaneously. After a crash, open `drive_log.txt` and search for the last few lines before the process exited.

Check the system journal:

```
journalctl -n 200 --no-pager
```

This shows the last 200 lines of the system log including kernel messages, which can reveal hardware issues that don't surface in Python output.

Add temporary print statements.

When you can't figure out where a crash is occurring, add `print()` statements at the start of each function you suspect. This is low-tech but reliable. Ask your agent: "Add print statements to this function to help me trace execution and identify where it's failing." Remove them before committing.

Using Your Agent for Rubber Duck Debugging

Sometimes you don't have an error — you have unexpected behavior. The car drives but pulls left. The model trained but performs worse than last week. The web interface loads but the camera feed is frozen.

These are harder to debug because there's no traceback to paste. The most effective approach is rubber duck debugging with your agent: describe the situation as precisely and completely as possible, as if explaining it to a colleague who knows nothing about your specific setup.

A good rubber duck prompt has four parts:

"Here's what I expected to happen: [describe expected behavior]. Here's what actually happened: [describe observed behavior]. Here's what I changed since it last worked correctly: [list every change]. Here's the relevant code and terminal output: [paste both]."

The act of writing this prompt carefully often surfaces the answer before the agent responds. When it doesn't, the agent has exactly the information it needs to generate useful hypotheses rather than generic suggestions.

Real Example: Debugging ROS 2 Hardware Integration with Claude Code

The best way to understand how to use an AI coding agent for robotics debugging is to see it applied to a real problem. This section walks through a representative debugging session from actual development work on a UCSD robotics project — setting up a ROS 2 stack on a Raspberry Pi with a VESC motor controller, OAK-D Lite camera, and Cartographer SLAM.

This is not a cleaned-up success story. It's a realistic account of the kind of multi-layer debugging that robotics projects actually involve, and how Claude Code was used at each stage.

The Stack

The project involved the following hardware and software:

- Raspberry Pi 5 running Ubuntu 24.04
- VESC motor controller connected over USB serial
- OAK-D Lite depth camera
- Joystick controller
- ROS 2 Jazzy
- Cartographer SLAM for mapping
- DonkeyCar framework for vehicle control

Any one of these components is manageable in isolation. When they all need to work together on the same Pi, with the correct Python environment, correct ROS 2 workspace, and correct device permissions, the failure space becomes large very quickly.

Problem 1: Cartographer SLAM Launch File Errors

What happened:

After getting ROS 2 installed, the next step was setting up Cartographer SLAM for mapping. The launch file from a reference implementation failed with an error referencing a parameter that didn't exist in the installed version of Cartographer.

The error:

```
[cartographer_node-1] terminate called after throwing an instance of  
'std::invalid_argument'  
  
[cartographer_node-1] what(): /home/pi/ros2_ws/src/cartographer_config/  
config/racecar_2d.lua:32: unknown field 'use_landmarks'
```

What was pasted into Claude Code:

The full traceback, the launch file, and the Lua configuration file referenced in the error. The configuration file had been copied from a tutorial written for a different version of Cartographer.

What Claude Code did:

It identified that `use_landmarks` was a parameter introduced in a later version of Cartographer than what was installed on the Pi. It then compared the configuration file against the parameter set supported by the installed version and flagged two additional parameters that would also fail — catching issues that hadn't surfaced yet because the node crashed before reaching them.

It produced a corrected version of the Lua configuration file with the incompatible parameters removed and added a comment explaining which parameters were version-specific, so future updates would be easier to manage.

What this illustrates:

Pasting the full configuration file alongside the error — not just the line that caused the crash — let Claude Code identify the broader pattern rather than just the immediate symptom. It caught two additional problems that would have become the next two debugging sessions.

Problem 2: VESC Serial Permission Error

What happened:

After getting SLAM running, the VESC motor controller needed to be integrated. The Python script that communicates with the VESC over USB serial failed immediately with:

`PermissionError: [Errno 13] Permission denied: '/dev/ttyACM0'`

What was pasted into Claude Code:

The error, the relevant section of the Python script, and the output of `ls -la /dev/ttyACM0`.

The `ls` output showed:

`crw-rw---- 1 root dialout 166, 0 Aug 12 14:23 /dev/ttyACM0`

What Claude Code did:

It immediately identified that the Pi's user wasn't in the `dialout` group, which owns serial devices on Ubuntu. It gave the fix and also explained why the error only appeared after connecting the VESC — the device file is created dynamically when the device is plugged in, so the permission issue was invisible until that point.

The fix:

```
sudo usermod -aG dialout $USER
```

Log out and reconnect for the group change to take effect.

What this illustrates:

This is a classic Linux permissions issue that affects almost every robotics project using USB serial devices. It's fast to fix once you know what it is, but if you don't know to look at group membership, you can spend a long time staring at a Python script that looks completely correct. Pasting the `ls -la` output alongside the error gave Claude Code the information it needed to diagnose it immediately.

Problem 3: OAK-D Lite Camera Not Detected After ROS 2 Launch

What happened:

The OAK-D Lite camera worked fine when tested with the standalone `depthai` Python library, but failed to initialize when launched through the ROS 2 node. The error was vague:

```
[depthai_ros_driver-2] [ERROR] [1723478234.521]: Failed to find device  
after 30.0s timeout
```

What was pasted into Claude Code:

The error, the launch file, and the output of `lsusb` showing the camera was physically detected by the Pi.

What Claude Code diagnosed:

Two possible causes ordered by likelihood:

1. The ROS 2 node was being launched as a different user than the one that had USB access permissions to the OAK-D device — the same class of problem as the VESC serial issue, but for USB rather than serial.
2. The `depthai` library version installed in the ROS 2 workspace didn't match the one installed system-wide, causing a version mismatch when the node tried to initialize the camera.

Verification steps it suggested:

For hypothesis 1: check `ls -la /dev/bus/usb/` and compare the group ownership to the groups the ROS launch user belongs to.

For hypothesis 2: compare `python3 -c "import depthai; print(depthai.__version__)"` run as the Pi user versus inside the ROS 2 workspace environment.

What happened:

Hypothesis 2 was correct. The system-wide `depthai` was version 2.24, but the ROS 2 workspace had sourced an older version. Pinning the workspace to the correct version resolved it.

What this illustrates:

When there's no clean traceback — just a timeout error — the hypothesis-first approach is more valuable than ever. Claude Code's suggestion to check both possibilities and verify each one independently prevented a long session of randomly trying fixes.

Patterns Across All Four Problems

Looking across these four debugging sessions, a few patterns emerge that apply to any robotics project:

The relevant information is almost never just the last line of the error. In every case above, the diagnosis depended on context that was several lines up in the output, or in a separate file entirely. Get in the habit of pasting more than you think is necessary.

Hardware and software errors look identical at the surface. The VESC permission error and the OAK-D detection failure both presented as vague Python errors. The actual cause in both cases was the Linux permission model, not the Python code. Running quick hardware state checks — `ls -la /dev/ttyACM*`, `lsusb`, `groups` — before diving into code saves time.

AI agents are best at narrowing the hypothesis space, not guessing the answer. In the Cartographer case, Claude Code caught three problems from one configuration file. In the OAK-D case, it gave two ordered hypotheses with specific verification steps. In neither case did it magically know the answer — it systematically identified what to look at next.

Ask for verification steps, not just fixes. The prompt "before suggesting a fix, give me the possible root causes and how to verify each one" consistently produced better results than "how do I fix this?" It forces a diagnostic step and gives you a mental model of the problem rather than just a command to run.